Predicting Coronary Heart Disease from the BRFSS Survey with Linear Models: Cleaning Matters More than the Model

Mario [Surname], Giovanni [Surname], Matteo Barbone
CS-433 Machine Learning, EPFL, Switzerland

***Abstract*—We predict whether a respondent of the 2015 Behavioral Risk Factor Surveillance System (BRFSS) has coronary heart disease (MICHD) from 321 self-reported survey answers, using only NumPy implementations of linear methods. Most of the 45% missing values encode the structure of the questionnaire, so we built a rule-based cleaning pipeline that yields 95 variables and a full-rank matrix of 256 features. A logistic regression trained by gradient descent, with a decision threshold tuned for F1, reaches F1 $= 0.4275 \pm 0.0019$ in 5-fold cross-validation. Ablations show that threshold tuning and data cleaning account for most of the performance, while interaction features do not help the logistic model.**

I. Introduction

The training set contains 328,135 respondents and 321 features; the test set 109,379 respondents. Only 8.8% of the respondents are positive, so accuracy is misleading: predicting "healthy" for everybody gives 91.2% accuracy and F1 $= 0$, and predicting "sick" for everybody gives F1 $= 0.162$. We therefore evaluate with the F1 score of the positive class, which is also the metric of the competition. Only NumPy is allowed, so every step is implemented from scratch.

II. Models and Methods

A. Understanding the missing values

45% of the cells are missing (NaN). Before imputing we looked for the *cause* of each NaN: (1) landline vs. cell-phone versions of the questionnaire; (2) interrupted interviews (15% of rows); (3) follow-up questions asked only after a "yes" (drinks are asked only to drinkers); (4) optional modules chosen by each state. In addition, the BRFSS uses special codes inside the numeric columns: $7/77/777$ = "don't know", $9/99/999$ = "refused", $88/888/555$ = "none". Left as numbers, a linear model reads them as huge quantities.

B. Cleaning (fixed rules, 321 → 95 variables)

Every data-dependent rule is computed on the *training* set and applied unchanged to the test set.

- **Removed a priori:** identifiers, dates, call-management variables, survey weights, landline/cell-phone variables (their NaN only reveals the phone type), and `HAREHAB1` (cardiac rehabilitation), which is filled in only by positive respondents and would leak the label.
- **Special codes:** "none" → 0; "don't know/refused" → NaN.
- **Structural zeros:** when a parent question says "no", the unasked quantity is 0 (non-drinkers: 0 drinks). The age at diabetes diagnosis becomes *years with diabetes* (0 for non-diabetics).
- **Sparse columns:** variables with more than 60% NaN are dropped. Keeping the optional modules changed F1 by only $+0.003$ to $+0.004$ (within the fold noise) and their weights mostly encoded "this state did not ask the module", a geographic proxy rather than health information. We keep only insulin use, merged with diabetes status.
- **Redundant columns:** we measure the *purity* of a pair (A, B), the fraction of rows where B takes the most frequent value associated with the value of A. Purity $= 1$ means B is a deterministic function of A; in that case B is dropped. Raw answers mixing units in one number ($1xx$/week, $2xx$/month) are replaced by the CDC-computed versions.
- **Other fixes:** remaining skip-logic NaNs are set to the implied answer (e.g. not hypertensive ⇒ no blood-pressure medication); four columns that are exact sums of others are removed; physically impossible values (e.g. more than 10,080 minutes of activity per week) become NaN.

C. Feature processing (95 variables → 256 features)

These statistics are learned on the training fold only:

- **34 continuous variables:** NaN replaced by the training median, plus a binary *missing indicator* (one per group of columns that are missing in exactly the same rows, 33 indicators); $\log(1+x)$ on the 21 long-tailed variables (alcohol, fruit and vegetables, physical activity); z-score standardization.
- **61 categorical variables:** one-hot encoding, with NaN treated as its own category, because missingness is often informative (e.g. 6.9% vs. 22% positives). The most frequent level is the reference and is left out.
- A bias column is added, and constant or identical columns are removed automatically.

A rank check of $X^\top X$ initially revealed 17 exact collinearities, all fixed by recoding (insulin, arthritis limitations) or dropping duplicated columns. The final matrix has full

Table I
REQUIRED METHODS, 5-FOLD CV, 256 FEATURES, F1-TUNED THRESHOLD. GD/SGD/LOGISTIC: 200 STEPS (SGD: ONE EPOCH), LAST ROW 1000 STEPS; S = TRAINING TIME.

Method	s	F1 val	prec.	rec.	acc.
least squares	0.3	0.4201 ± 0.0023	0.350	0.526	0.872
ridge	0.5	0.4201 ± 0.0023	0.350	0.525	0.872
GD (MSE)	9.5	0.4165 ± 0.0019	0.334	0.553	0.863
SGD (MSE)	6.7	0.4053 ± 0.0050	0.329	0.528	0.863
logistic, $4/L$	11.2	0.4000 ± 0.0028	0.327	0.516	0.863
reg. logistic, $4/L$	11.4	0.4005 ± 0.0028	0.325	0.522	0.862
logistic, $16/L$	56	$\mathbf{0.4275 \pm 0.0019}$	0.348	0.553	0.869

rank (condition number 7×10^4), which is required by `least_squares`.

D. Models, step size and threshold

We use the six required methods with labels $y \in \{0, 1\}$. Linear models output the score $s = x^\top w$, logistic models $s = \sigma(x^\top w)$. The gradient-descent step is set from L, the largest eigenvalue of $X^\top X/N$ (power method, $L \approx 8.3$): $\gamma = 1/L$ for MSE. The logistic loss has curvature at most $L/4$, so $\gamma = 4/L$ is guaranteed to converge; in practice $\gamma = 16/L$ converges faster and $\gamma = 64/L$ diverges: on fold 1, after 1000 steps the training loss is 0.2227 ($4/L$) vs. 0.2209 ($16/L$), while with $64/L$ it oscillates (0.99 after 300 steps). The classifier predicts positive when $s \geq t$, where the threshold t maximizes F1 *on the training fold* (grid of 200 percentiles of s).

E. Evaluation protocol

Stratified 5-fold cross-validation (same positive rate 8.83% in each fold, seed 1), with all preprocessing refitted inside each fold. We report mean $\pm$ standard deviation over the folds. Two variants are compared on the same folds: a difference is considered real only if the mean of the per-fold differences is larger than their standard deviation.

III. RESULTS

Methods (Table I). The closed-form methods reach their optimum immediately; GD and SGD on the MSE get close to it. With 200 steps the logistic regression is not converged (validation F1 0.386 $\rightarrow$ 0.424 from 100 to 1000 steps at $4/L$); with $\gamma = 16/L$ and 1000 steps it becomes the best model. The paired difference with least squares is $+0.0073$, positive in all five folds (std 0.0018). Regularization changes nothing: with 328k rows for 256 features there is no overfitting (training F1 0.431 vs. validation 0.4275).

Ablations (Table II). Tuning the threshold is by far the most important choice: with $t = 0.5$ the model is very conservative (precision 0.56, recall 0.15) and F1 drops by 0.19. The cleaning is the second one: a naive pipeline on the raw columns (special codes left as numbers, $> 60\%$-NaN columns dropped, mean imputation, z-score) loses 0.015 in every fold. The individual encoding choices matter much less. Imputing the categorical NaNs with the mode gives the same F1 with 202 instead of 256 features, and removing the missing indicators changes nothing beyond noise, because most of the missingness information is already in the NaN levels of the categoricals. The $\log(1 + x)$ transform, introduced to reduce skewness, slightly *hurts* (-0.0009, consistent over the 5 folds but negligible); we found this after freezing the submitted model and kept it to keep `run.py` identical to the submission.

Table II
ABLATION STUDY (LOGISTIC REGRESSION, 1000 IT., $16/L$, 5-FOLD CV). Δ = PAIRED DIFFERENCE WITH THE FINAL MODEL.

Variant	F1 val	Δ F1
final model (256 feat.)	$\mathbf{0.4275 \pm 0.0019}$	–
threshold $t = 0.5$	0.2363 ± 0.0054	-0.1912 ± 0.0054
naive pipeline (193 feat.)	0.4122 ± 0.0020	-0.0153 ± 0.0017
categ. NaN $\rightarrow$ mode (202)	0.4275 ± 0.0029	$+0.0000 \pm 0.0020$
no missing indicators (225)	0.4279 ± 0.0024	$+0.0005 \pm 0.0010$
no $\log(1 + x)$	0.4284 ± 0.0022	$+0.0009 \pm 0.0006$
+ data interactions (300)	0.4272 ± 0.0021	-0.0003 ± 0.0010
+ medical interactions (279)	0.4272 ± 0.0025	-0.0003 ± 0.0015
least sq. + medical inter.	0.4245 ± 0.0029	$+0.0044^*$

* w.r.t. least squares without interactions (positive in all folds).

Interactions. Since cardiac risk multiplies (smoking matters more at 70 than at 30), we added pairwise products between the 10 most important features (importance $|w_j| \operatorname{std}(x_j)$, chosen inside each fold) or between medical pairs (age $\times$ sex/diabetes/smoking/health, diabetes $\times$ hypertension, hypertension $\times$ cholesterol). They help least squares, whose linear fit of a 0/1 label has the wrong shape, but not the logistic regression. Since they do not even raise the *training* F1 of the logistic model (0.431), the model is not too simple: the limit is the information in self-reported survey answers.

Test set. The final model, retrained on the full training set (threshold 0.195, 14% of the test set predicted positive, consistent with recall/precision $\approx 0.55/0.35$), obtains F1 = **[AIcrowd F1]** and accuracy = **[AIcrowd accuracy]** on AIcrowd.

IV. CONCLUSION

The most useful decisions concerned the data, not the model: understanding *why* values are missing turned a 45%-missing dataset into a clean full-rank matrix, and tuning the threshold for F1 matters far more than the choice among linear models. The tiny train/validation gap and the failure of interactions to raise it point to label and feature noise (self-reported diagnoses, no clinical measurements), not underfitting. **Code:** `run.py` reproduces the submission from the raw CSVs in about one minute (deterministic); `cleaning.py`/`preprocessing.py` hold the cleaning rules and feature processing, `implementations.py` the six methods, the notebooks the CV and ablations.